Aftermath Market Making

Security Assessment

January 25th, 2025 — Prepared by OtterSec

Sangsoo Kang sangsoo@osec.io

Michał Bochnak embe221ed@osec.io

Robert Chen r@osec.io

Table of Contents

Executive Summary **3**

Overview 3

Key Findings 3

Scope **4**

Findings **5**

Vulnerabilities **6**

OS-AMM-ADV-00 | Lack of Access Control 8

OS-AMM-ADV-01 | Tolerance Check Bypass on Forced Withdrawal 9

OS-AMM-ADV-02 | Fund Loss Due to Unchecked Conversion 10

OS-AMM-ADV-03 | Fee Manipulation via Improper LP Coin Split 11

OS-AMM-ADV-04 | Miscalculation Due to Negative Withdrawal Amount 13

OS-AMM-ADV-05 | DoS Due to Surpassing of Event Limit 14

OS-AMM-ADV-06 | Risk of Negative Margin Calculation 15

OS-AMM-ADV-07 | Improper Zero Mint Check 16

OS-AMM-ADV-08 | Inability to Withdraw Owner Fees 17

OS-AMM-ADV-09 | Misalignment of Token Metadata 18

OS-AMM-ADV-10 | Failure to Update Last Used Timestamp of Vault Cap 19

General Findings **20**

OS-AMM-SUG-00 | Utilization of Incorrect Relational Operator 21

OS-AMM-SUG-01 | Misleading Event Emission 23

OS-AMM-SUG-02 | Missing Validation Logic 24

OS-AMM-SUG-03 | Code Maturity 25

© 2025 Otter Audits LLC. All Rights Reserved.

TABLE OF CONTENTS

OS-AMM-SUG-04 | Code Refactoring 26

Appendices

Vulnerability Rating Scale **27**

Procedure **28**

© 2025 Otter Audits LLC. All Rights Reserved.

01 — Executive Summary

Overview

Aftermath engaged OtterSec to assess the `market-making-v2` program. This assessment was conducted between January 6th and January 20th, 2025. For more information on our auditing methodology, refer to Appendix B.

Key Findings

We produced 16 findings throughout this audit engagement.

In particular, we identified a critical vulnerability involving the exploitation of a high minimum expected balance output amount to trigger the force withdrawal mechanism, which bypasses margin tolerance checks when closing all positions with market orders, thereby exposing the system to significant slippage risks (OS-AMM-ADV-01). We also identified another issue wherein if the accumulated slippage exceeds the withdrawal balance, it creates a negative value that is then converted to an absolute amount, allowing attackers to gain unexpected profits at the vault's expense (OS-AMM-ADV-02).

We further highlighted the lack of any functionality for withdrawing accumulated owner fees, leaving collected fees inaccessible and potentially locking significant funds (OS-AMM-ADV-08), and the possibility of unauthorized interactions with perpetuities, as the account allows direct access to the trading account (OS-AMM-ADV-00). Additionally, if a short position incurs losses larger than amount to withdraw, the subsequent calculation produces a negative value, resulting in miscalculations (OS-AMM-ADV-04).

Furthermore, the vault owner may accumulate excessive pending orders across multiple clearing houses, surpassing the maximum event emission limit and blocking the force withdrawal process. Similarly, an attacker may flood the order book with numerous small orders, also exceeding the event emission limit (OS-AMM-ADV-05).

We also made suggestions regarding inconsistencies in the codebase and ensuring adherence to coding best practices (OS-AMM-SUG-03) and advised to avoid emission of misleading events (OS-AMM-SUG-01). We further recommended implementing validation to verify the new version is strictly greater than the current version, and enforcing checks to ensure stop order tickets do not interact with the clearing house during active deposit or withdraw sessions (OS-AMM-SUG-02).

© 2025 Otter Audits LLC. All Rights Reserved.

02 — Scope

The source code was delivered to us in a Git repository at https://github.com/AftermathFinance/market-making-v2. This audit was performed against commit f1c5aa9 and 16dc86e.

A brief description of the programs is as follows:

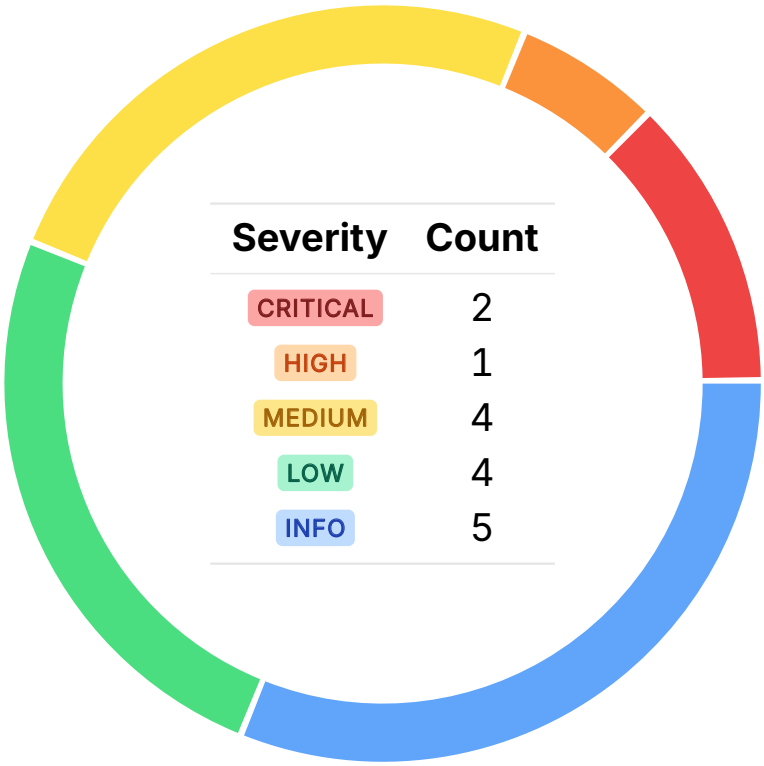
Name	Description
market-making-v2	The market making module interacts with Aftermath Perpetual contracts, enabling vault operators to manage liquidity provided by participants. Profits and losses are shared based on the liquidity contributed.

© 2025 Otter Audits LLC. All Rights Reserved.

03 — Findings

Overall, we reported 16 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.



Severity	Count
CRITICAL	2
HIGH	1
MEDIUM	4
LOW	4
INFO	5

© 2025 Otter Audits LLC. All Rights Reserved.

04 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities we identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in Appendix A.

ID	Severity	Status	Description
OS-AMM-ADV-00	CRITICAL	RESOLVED	`vault::account` allows direct access to the trading account, enabling unauthorized interactions with a perpetual.
OS-AMM-ADV-01	CRITICAL	RESOLVED	A user may exploit high `min_expected_balance_out` values to trigger the force withdrawal mechanism, which bypasses margin tolerance checks when closing all positions utilizing market orders, exposing the system to significant slippage risks.
OS-AMM-ADV-02	HIGH	RESOLVED	If accumulated slippage exceeds the withdrawal balance, it creates a negative value that is converted to an absolute amount, allowing attackers to gain unexpected profits at the vault's expense.
OS-AMM-ADV-03	MEDIUM	RESOLVED	`vault::split_user_lp_coin` allows for creating a `UserLpCoin` with zero provided `USD` value and non-zero LP balance, potentially manipulating fee calculations during withdrawals in `end_withdraw_session`.
OS-AMM-ADV-04	MEDIUM	RESOLVED	If a short position incurs losses larger than `amount_to_withdraw`, it causes the calculation to produce a negative value, resulting in miscalculations.

© 2025 Otter Audits LLC. All Rights Reserved.

OS-AMM-ADV-05	MEDIUM	RESOLVED	The vault owner may accumulate excessive pending orders across multiple clearing houses, surpassing the maximum event emission limit and blocking the force withdrawal process. Similarly, an attacker could flood the order book with numerous small orders, also exceeding the event emission limit.
OS-AMM-ADV-06	MEDIUM	RESOLVED	`get_vault_margin_in_market` may return a negative margin during deposit and withdrawal sessions, resulting in incorrect margin calculations and unsafe liquidity conditions for the vault.
OS-AMM-ADV-07	LOW	RESOLVED	The `lp_to_mint != 0` check in `end_deposit_session` does not accurately prevent minting zero tokens, as the `to_balance` conversion may still result in zero despite the non-zero value of `lp_to_mint`.
OS-AMM-ADV-08	LOW	RESOLVED	The vault lacks a function for withdrawing accumulated owner fees, leaving collected fees inaccessible and potentially locking significant funds.
OS-AMM-ADV-09	LOW	RESOLVED	`create_vault_cap` incorrectly passes the `name` and `symbol` parameters in reverse order to `create_currency`, resulting in improper metadata assignment.
OS-AMM-ADV-10	LOW	RESOLVED	The failure to update `last_used_timestamp_ms` after modifying the fee rate, lock period, or force withdraw delay allows the cool-down period to be bypassed, enabling potential abuse by vault owners through repeated updates in quick succession.

© 2025 Otter Audits LLC. All Rights Reserved.

Lack of Access Control CRITICAL OS-AMM-ADV-00

Description

`vault::account` currently allows public access to a reference to the trading account associated with a vault. If a malicious actor accesses the account via `vault::account_mut`, they can modify trade positions or collateral balances, possibly leading to a loss of funds.

>_ *market-making-vault/sources/vault.move* RUST

```rust
/// Retrieves a reference to the trading account associated with the given Vault.
///
/// Parameters:
/// - `vault`: Reference to the `Vault` from which the trading account is being accessed.
///
/// Returns:
/// - `&Account<C>`: A reference to the `Account` associated with the specified Vault.
public fun account<L, C>(
    vault: &Vault<L, C>
): &Account<C> {
    // Borrow a reference to the trading account associated with the vault.
    dof::borrow(&vault.id, keys::account_key())
}
```

Remediation

Update the access control modifier to `public(package)` to prevent access to the function.

Patch

Resolved in a6316ee.

© 2025 Otter Audits LLC. All Rights Reserved.

Tolerance Check Bypass on Forced Withdrawal CRITICAL OS-AMM-ADV-01

Description

The vulnerability in `process_clearing_house_for_force_withdraw` arises from the possibility of bypassing tolerance checks by closing positions with market orders. Closing all bases allows the tolerance check on the margin ratio to be skipped. The skipped tolerance check (`force_withdraw_above_margin_ratio_tolerance`) is meant to ensure the withdrawal action keeps the account margin ratios within a safe range. By bypassing this, the function potentially exposes the account to greater losses due to slippage. Market orders execute trades immediately at the current best available prices. These orders do not control for slippage.

With the margin tolerance check bypassed when the base position is closed, losses are exacerbated since the position may not satisfy the safety thresholds for acceptable collateral allocation. This issue is compounded by the fact that a malicious user may intentionally set `min_expected_balance_out` to an unrealistically high value that the vault will not be able to satisfy. This effectively locks the withdrawal session. Consequently, the user may trigger a forced withdrawal, bypassing the default constraints on withdrawal processing, resulting in losses to the vault.

Remediation

Update `request_timestamp_ms` in `set_new_withdraw_request_slippage` to prevent users from intentionally delaying and perform a tolerance check when `base_amount` is greater than zero.

Patch

Resolved in 25bcdcb.

© 2025 Otter Audits LLC. All Rights Reserved.

Fund Loss Due to Unchecked Conversion HIGH OS-AMM-ADV-02

Description

The vulnerability concerns how `end_withdraw_session` calculates the `balance_to_withdraw`, processes accumulated slippage, and converts this value into a withdrawable balance. Specifically, when the absolute value of the `accumulated_slippage` exceeds the `balance_to_withdraw`. In this case, the result of `balance_to_withdraw` becomes negative, which is not a valid value for withdrawal, however as the negative value is then passed to `to_balance`, it is converted to an absolute (positive) value. Thus, an attacker may receive unexpected profit while causing a loss of funds for the vault.

>_ *market-making-vault/sources/vault.move* RUST

```rust
public(package) fun end_withdraw_session<L, C>(
    withdraw_session: WithdrawSession<L, C>,
    ctx: &mut TxContext
) {
    [...]
    let mut balance_to_withdraw = multiply_by_rational_ifixed(
        ifixed::from_balance(lp_balance.value(), constants::b9_scaling()),
        total_vault_balance,
        ifixed::from_balance(vault.lp_supply_value(), constants::b9_scaling()),
    );
    balance_to_withdraw = ifixed::add(balance_to_withdraw, accumulated_slippage);
    let lp_balance_value = lp_balance.value();
    burn_lp_balance(&mut vault, lp_balance);
    let mut withdrawn_balance = vault.remove_collateral_from_vault(
        ifixed::to_balance(
            ifixed::div(balance_to_withdraw, collateral_price),
            scaling_factor
            )
    );
    [...]
}
```

Remediation

Ensure that `balance_to_withdraw` is non-negative before passing it to `to_balance`.

Patch

Resolved in 66e5dfd.

© 2025 Otter Audits LLC. All Rights Reserved.

Fee Manipulation via Improper LP Coin Split MEDIUM OS-AMM-ADV-03

Description

`split_user_lp_coin` calculates a `split_ratio`, based on the amount the user wants to split relative to the total `lp_balance` of the original `UserLpCoin`. This `split_ratio` is applied to the `provided_value_usd` of the original `UserLpCoin`. If a new `UserLpCoin` is created after the split, the new `provided_value_usd` will be calculated as `provided_value_usd - new_provided_value_usd`. This results in a reduced `provided_value_usd` for the original coin and a new `UserLpCoin` with this new `provided_value_usd`.

>_ *market-making-vault/sources/vault.move* RUST

```rust
public(package) fun split_user_lp_coin<L>([...]): UserLpCoin<L> {
    // Calculate the split ratio to adjust the initial provided usd value
    let split_ratio = ifixed::div(
        ifixed::from_balance(amount, constants::b9_scaling()),
        ifixed::from_balance(user_lp_coin.lp_balance.value(), constants::b9_scaling())
    );
    let new_provided_value_usd = ifixed::mul(user_lp_coin.provided_value_usd,split_ratio);
    user_lp_coin.provided_value_usd = user_lp_coin.provided_value_usd -new_provided_value_usd;
    // Split balance and create new UserLpCoin
    UserLpCoin {
        id: object::new(ctx),
        lp_balance: user_lp_coin.lp_balance.split(amount),
        start_timestamp_ms: user_lp_coin.start_timestamp_ms,
        provided_value_usd: new_provided_value_usd
    }
}
```

Therefore, it is possible to create a new `UserLpCoin` object with a `provided_value_usd` of zero, even though the `lp_balance` is non-zero. This will result in the original `UserLpCoin` (the one from which the new coin was split) retaining its `provided_value_usd` while losing some of its `lp_balance`. `end_withdraw_session` utilizes the `provided_value_usd` to compute the fees and withdrawal amounts.

Thus, when the `UserLpCoin` with a non-zero LP balance but zero `provided_value_usd` is introduced, although this `UserLpCoin` has no contribution to the `provided_value_usd`, it is still entitled to a withdrawal of funds based on its LP balance. This withdrawal bypasses the fee calculation mechanism, since the fees are determined by the profits derived from `balance_to_withdraw` and `provided_value_usd`.

© 2025 Otter Audits LLC. All Rights Reserved.

Remediation

Ensure the `provided_value_usd` does not become zero.

Patch

Resolved in ffaff4b.

© 2025 Otter Audits LLC. All Rights Reserved.

Miscalculation Due to Negative Withdrawal Amount MEDIUM OS-AMM-ADV-04

Description

The `amount_to_withdraw` calculation in `process_clearing_house_for_force_withdraw` has a potential vulnerability that arises from an edge case where short positions incur a large loss. If the loss on the short position (negative PnL) exceeds the initial `amount_to_withdraw` calculated based on the user's LP share of the position's margin, the result of `ifixed::sub(pnl, fees)` will produce a large negative value. Adding this negative value to the `amount_to_withdraw` will cause the output to become negative. Thus, `amount_to_withdraw` becomes negative, resulting in miscalculations that impact both the user and the vault.

>_ *market-making-vault/sources/vault.move* RUST

```rust
public(package) fun process_clearing_house_for_force_withdraw<L, C>(
    [...]
) {
    [...]
    // Close the required part of the position. Apply slippage and fees to the amount
    // to withdraw for this force withdrawal
    if (base_amount != 0) {
       [...]
        let pnl = if (position_is_ask) {
            ifixed::sub(expected_quote_asset_delta, quote_asset_delta)
        } else {
            ifixed::sub(quote_asset_delta, expected_quote_asset_delta)
        };
        let fees = ifixed::mul(quote_asset_delta, taker_fee);
        withdraw_session.accumulated_slippage = ifixed::add(
            withdraw_session.accumulated_slippage,
            ifixed::sub(pnl, fees)
        );
        amount_to_withdraw = ifixed::add(amount_to_withdraw, ifixed::sub(pnl, fees));
    };
    [...]
}
```

Remediation

Add a condition to check and prevent `amount_to_withdraw` from becoming negative.

Patch

Resolved in a53cf8e.

© 2025 Otter Audits LLC. All Rights Reserved.

DoS Due to Surpassing of Event Limit MEDIUM OS-AMM-ADV-05

Description

The `max_num_event_emit` sets a restriction on how many events may be emitted at a time. A vault owner may intentionally place multiple pending orders across various clearing houses, thereby accumulating a large number of events to be processed. If the number of events exceeds 1024, it will block the force withdrawal process.

Remediation

Enforce tighter validation or limits on the number of pending orders a vault owner may place.

Patch

Resolved in d41ff62.

© 2025 Otter Audits LLC. All Rights Reserved.

Risk of Negative Margin Calculation MEDIUM OS-AMM-ADV-06

Description

The current implementation of `get_vault_margin_in_market` may return a negative margin in deposit and withdrawal sessions. This can cause `vault_balance_value` to become negative, leading to incorrect calculations in `end_session`. This happens because `lp_to_mint` becomes negative but is converted to its absolute value in `to_balance`, affecting the computation.

>_ *market-making-vault/sources/vault.move* RUST

```rust
        // Calculate the amount of lp_coin to mint for the user
        let lp_to_mint = multiply_by_rational_ifixed(
            provided_balance_value,
            ifixed::from_balance(vault.lp_supply_value(), constants::b9_scaling()),
            vault_balance_value
        );
        assert!(lp_to_mint != 0, errors::user_lp_calculation_zero());

        // Mint the lp_coin and check for slippage
        let lp_balance = mint_lp_balance(
            &mut vault,
            ifixed::to_balance(lp_to_mint, constants::b9_scaling())
        );
```

Remediation

Abort a transaction in case of a negative `vault_balance_value`.

Patch

Resolved in b041a2e.

© 2025 Otter Audits LLC. All Rights Reserved.

Improper Zero Mint Check LOW OS-AMM-ADV-07

Description

In `end_deposit_session`, the check `assert!(lp_to_mint != 0)` only ensures that the `lp_to_mint` value, as calculated before the conversion to a fixed-point format (`to_balance`), is non-zero. However, the conversion process itself (specifically the call to `ifixed::to_balance`) may still result in a minted amount of zero liquidity provider (LP) tokens, even if the value of `lp_to_mint` is non-zero prior to conversion.

>_ *market-making-vault/sources/vault.move* RUST

```rust
public(package) fun end_deposit_session<L, C>(
    deposit_session: DepositSession<L, C>,
    min_expected_lp_coin_out: u64,
    ctx: &mut TxContext
) {
    [...]
    assert!(lp_to_mint != 0, errors::user_lp_calculation_zero());

    // Mint the lp_coin and check for slippage
    let lp_balance = mint_lp_balance(
        &mut vault,
        ifixed::to_balance(lp_to_mint, constants::b9_scaling())
    );
    [...]
}
```

Remediation

Perform a validation after the conversion to ensure the minted LP balance is valid and non-zero.

Patch

Resolved in 25bcdcb.

© 2025 Otter Audits LLC. All Rights Reserved.

Inability to Withdraw Owner Fees LOW OS-AMM-ADV-08

Description

The vault allows for the collection of owner fees, however, it currently lacks a function to withdraw these accumulated fees, introducing operational inefficiencies. Without a withdrawal function, owner fees remain inaccessible, causing substantial funds to be locked in the contract over time as they accumulate, which negatively impacts the protocol's revenue model.

Remediation

Implement a function that enables the vault owner to withdraw the accumulated fees.

Patch

Resolved in 415146d.

© 2025 Otter Audits LLC. All Rights Reserved.

Misalignment of Token Metadata LOW OS-AMM-ADV-09

Description

The issue arises from incorrect parameter ordering when calling `coin::create_currency` within `create_vault_cap`. `create_currency` defines the expected order of parameters as [`symbol`, `name`]. However, `create_vault_cap` passes the `name` parameter in place of `symbol` and the `symbol` parameter instead of `name`.

>_ *market-making-vault/sources/vault.move* RUST

```rust
public(package) fun create_vault_cap<L: drop>(
    [...]
) {
    let (treasury_cap, mut coin_metadata) = coin::create_currency<L>(
        witness,
        decimals,
        name,
        symbol,
        description,
        option::none(),
        ctx
    );
    [...]
}
```

As a result, the shorthand `symbol` and full `name` are reversed during token creation. A mismatch in expected token metadata will create confusion among users and external programs interacting with the token affecting the operational integrity of the program.

Remediation

Update `create_vault_cap` to pass the parameters in the correct order to `create_currency`.

Patch

Resolved in 25bcdcb.

© 2025 Otter Audits LLC. All Rights Reserved.

Failure to Update Last Used Timestamp of Vault Cap LOW OS-AMM-ADV-10

Description

The issue concerns the lack of updates to the `vault_owner_cap.last_used_timestamp_ms` field within `update_owner_fee_rate`, `update_lock_period`, and `update_force_withdraw_delay`. The cooldown, implemented by comparing `vault_owner_cap.last_used_timestamp_ms` with the current `clock.timestamp_ms()`, is intended to restrict updates to parameters such as `owner_fee_rate`, `lock_period`, and `force_withdraw_delay` to occur only after a specified time interval (`vault_params_update_frequency()`) has passed.

However, due to the failure to update `last_used_timestamp_ms` after successfully modifying the vault parameters, the recorded last-used timestamp in the `VaultOwnerCap` remains unchanged, allowing additional updates to bypass the cool-down check. As a result, it will be possible to continuously update these parameters to extremely low or high values, destabilizing the system.

Remediation

Ensure `update_owner_fee_rate`, `update_lock_period`, and `update_force_withdraw_delay` update the `vault_owner_cap.last_used_timestamp_ms` field after successfully modifying the respective parameter.

Patch

Resolved in db6c2f1.

© 2025 Otter Audits LLC. All Rights Reserved.

05 — General Findings

Here, we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-AMM-SUG-00	Preventing the addition of extra elements beyond predefined limits by ensuring checks utilize `<` rather than `<=`.
OS-AMM-SUG-01	The `can_force_process` flag allows vault owners to manipulate withdrawal sessions via misleading event emissions by processing standard withdrawals while emitting force withdrawal events.
OS-AMM-SUG-02	`update_vault_version` lacks proper validation of the `VaultOwnerCap` against the target vault and fails to ensure new version is strictly greater than the current version.
OS-AMM-SUG-03	Suggestions regarding inconsistencies in the codebase and ensuring adherence to coding best practices.
OS-AMM-SUG-04	Recommendations for modifying the codebase to mitigate potential security issues and improve functionality.

© 2025 Otter Audits LLC. All Rights Reserved.

Utilization of Incorrect Relational Operator OS-AMM-SUG-00

Description

`deposit_into_perpetuals` and `place_limit_order` currently utilize `<=` in the conditions for verifying pending orders and clearing house IDs, which is inconsistent with the subsequent login in these functions. The assertion in `deposit_into_perpetuals` verifies that number of clearing house IDs in the vault is equal to or fewer than its `max_markets_in_vault` limit. This implies the vault may already contain `max_markets_in_vault` clearing house IDs. However, a new clearing house ID is appended subsequently, potentially causing this limit to be exceeded.

>_ *market-making-vault/sources/perpetuals_api.move* RUST

```rust
public(package) fun deposit_into_perpetuals<L, C>([...]) {
    [...]
    let clearing_house_id = object::id(clearing_house);
    if(!vector::contains(&vault.ch_ids(), &clearing_house_id)) {
        assert!(vector::length(&vault.ch_ids()) <=
            ↪  vault.max_markets_in_vault(),errors::max_markets_exceeded());
        vector::push_back(vault.ch_ids_mut(), clearing_house_id);
        let vault_account = vault.account_mut();
        let account_id = vault_account.get_account_id();
        if (!clearing_house.exists_position(account_id)) {
            perpetuals::create_market_position(clearing_house, vault_account);
        }
    };
    [...]
}
```

Similarly, in `place_limit_order` asserts `get_pending_orders_counter(position) <= vault.max_pending_orders_per_position`. Thus, if there are already `max_pending_orders_per_position` orders pending, placing a new limit order will exceed the allowed limit.

>_ *market-making-vault/sources/perpetuals_api.move* RUST

```rust
public(package) fun place_limit_order<L, C>([...]) {
    [...]
    assert!(position::get_pending_orders_counter(position)
        ↪  <=vault.max_pending_orders_per_position(), errors::max_pending_orders_exceeded());
    [...]
    // i. Start a new session and place the limit order.
    let mut session = perpetuals::start_session(
        clearing_house,
        vault.account_mut(),
```

© 2025 Otter Audits LLC. All Rights Reserved.

```
        base_oracle,
        collateral_oracle,
        clock,
    );
    perpetuals::place_limit_order(&mut session, side, size, price, order_type);
    [...]
}
```

Remediation

Update `deposit_into_perpetuals` and `place_limit_order` to utilize `<` instead of `<=` to ensure that the operations will proceed only if adding the next element will not exceed the maximum allowed limit.

Patch

Resolved in d41ff62.

© 2025 Otter Audits LLC. All Rights Reserved.

Misleading Event Emission

OS-AMM-SUG-01

Description

`vault::end_withdraw_session` checks the `can_force_process` flag to determine whether the withdrawal session is a force withdrawal or standard withdrawal. The function emits a `UserForceWithdraw` event if `can_force_process` is true, and an `OwnerWithdraw` event if `can_force_process` is false. However, as the vault owner has control over the withdrawal process and their own `UserLpCoin`, they may set the `can_force_process` flag to true, yet proceed to handle the withdrawal request in the standard way instead of the force withdrawal method. As a result, the function emits the `UserForceWithdraw` event, even though the withdrawal was not processed as a forced one.

Remediation

Ensure that if `can_force_process` is set to true, the withdrawal must adhere to forced withdrawal requirements.

Patch

Resolved in 6d78b43.

© 2025 Otter Audits LLC. All Rights Reserved.

Missing Validation Logic

OS-AMM-SUG-02

Description

1. Currently, in `vault::update_vault_version` utilizes the `VaultOwnerCap` as a parameter but does not validate its connection to the specified vault. Either ensure that the provided `VaultOwnerCap` is explicitly tied to the updated `vault`, or modify the function to allow any user to trigger the version update without ownership verification. Furthermore, validate whether the new version is greater than the previous version to maintain consistency, security, and versioning integrity within the system.

>_ *market-making-vault/sources/vault.move* RUST

```rust
public(package) fun update_vault_version<L, C>(
    _vault_owner_cap: &VaultOwnerCap,
    vault: &mut Vault<L, C>
) {
    // Update the vault's version to the latest version specified in the constants module.
    vault.version = constants::version();
    events::emit_update_vault_version(
        vault.id(),
        constants::version(),
    );
}
```

2. In `update_max_pending_orders_per_position`, ensure the new limit is not lower than the highest number of existing pending orders across all positions, and in `update_max_markets_in_vault`, check if the new limit is below the number of markets already registered in the vault.

Patch

1. Issue #1 resolved in dbd0db8.

Remediation

Add the above validations.

© 2025 Otter Audits LLC. All Rights Reserved.

Code Maturity OS-AMM-SUG-03

Description

1. Presently the codebase utilizes a mix of two function calling conventions: `obj.foo()` and `module::foo(obj)`. To maintain consistency, it is recommended to standardize on a single convention.
2. `create_vault_cap` currently determines the `effective_icon_url` after calling `coin::create_currency` and updates the token's metadata through a separate `coin::update_icon_url` call. To simplify the code, it would be appropriate to package `effective_icon_url` in an `option::some` and pass it directly to `create_currency`.

   >_ *market-making-vault/sources/vault.move* RUST

   ```rust
   public(package) fun create_vault_cap<L: drop>(
       [...]
   ) {
       let (treasury_cap, mut coin_metadata) = coin::create_currency<L>(
           [...]
       );
       let effective_icon_url = if (!vector::is_empty(&icon_url)) {
           icon_url
       } else {
           constants::default_lp_coin_image()
       };
       coin::update_icon_url(&treasury_cap, &mut coin_metadata,
           ↪  ascii::string(effective_icon_url));
       [...]
   }
   ```

3. Some docstrings appear outdated, such as the parameter list in `create_vault`. Ensure all docstrings are updated.

Remediation

Implement the above-mentioned suggestions.

© 2025 Otter Audits LLC. All Rights Reserved.

Code Refactoring

OS-AMM-SUG-04

Description

1. `update_max_pending_orders_per_position` and `update_max_markets_in_vault` should emit events similar to other update functions to ensure proper logging of state changes.

Remediation

Incorporate the above refactors.

Patch

1. Issue #1 resolved in d4b52ef.

© 2025 Otter Audits LLC. All Rights Reserved.

A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the General Findings.

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
- Improperly designed economic incentives leading to loss of funds.

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
- Exploitation involving high capital requirement with respect to payout.

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
- Forced exceptions in the normal user flow.

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
- Improved input validation.

© 2025 Otter Audits LLC. All Rights Reserved.

B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.

© 2025 Otter Audits LLC. All Rights Reserved.